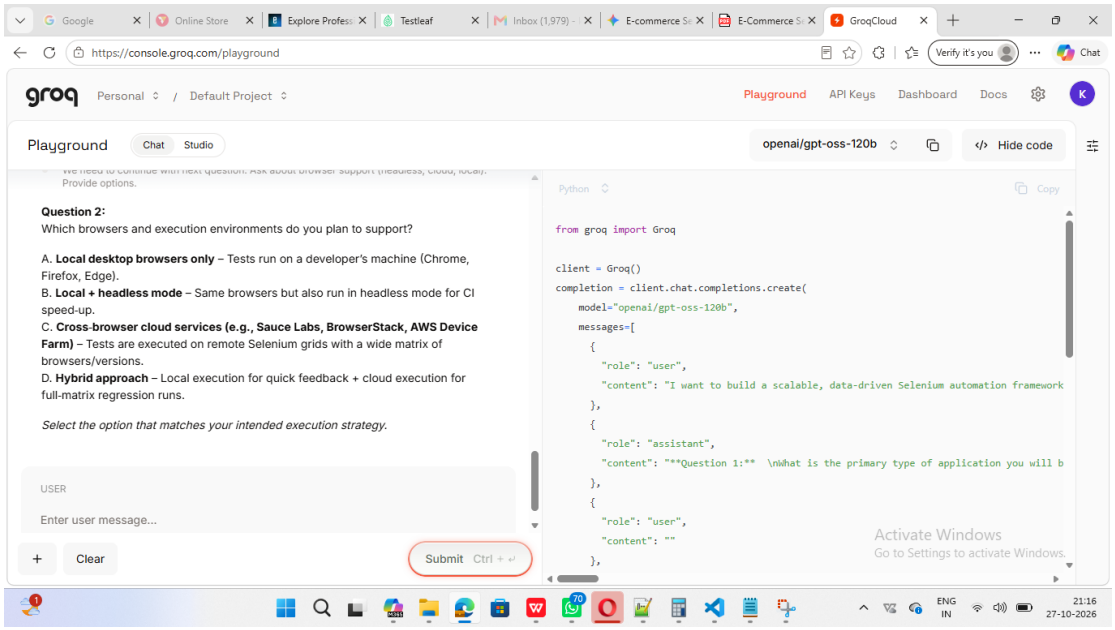
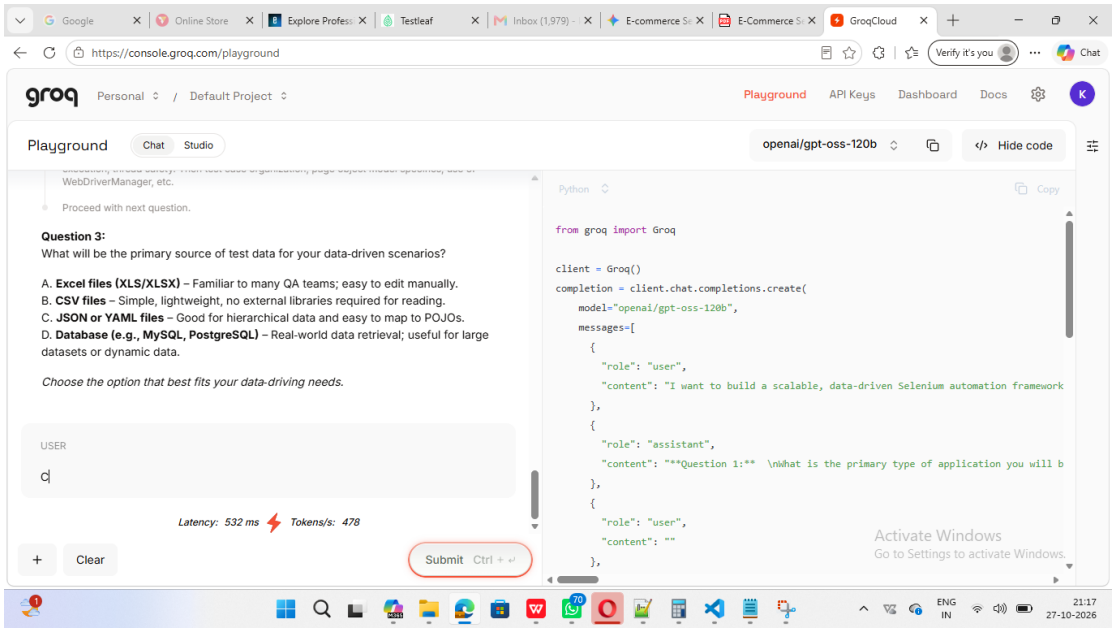
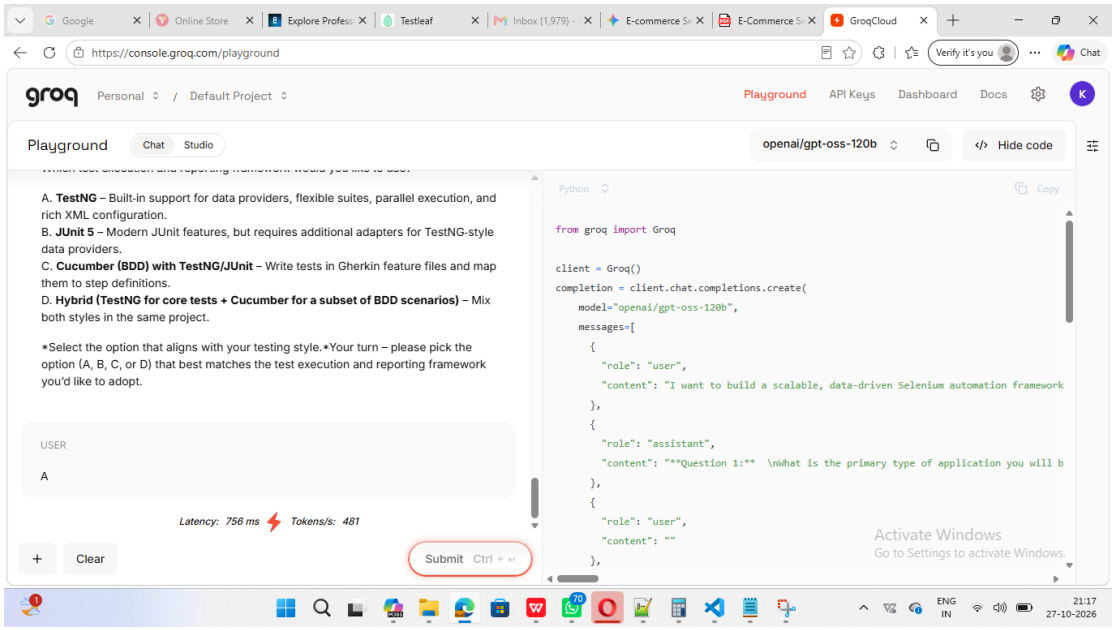
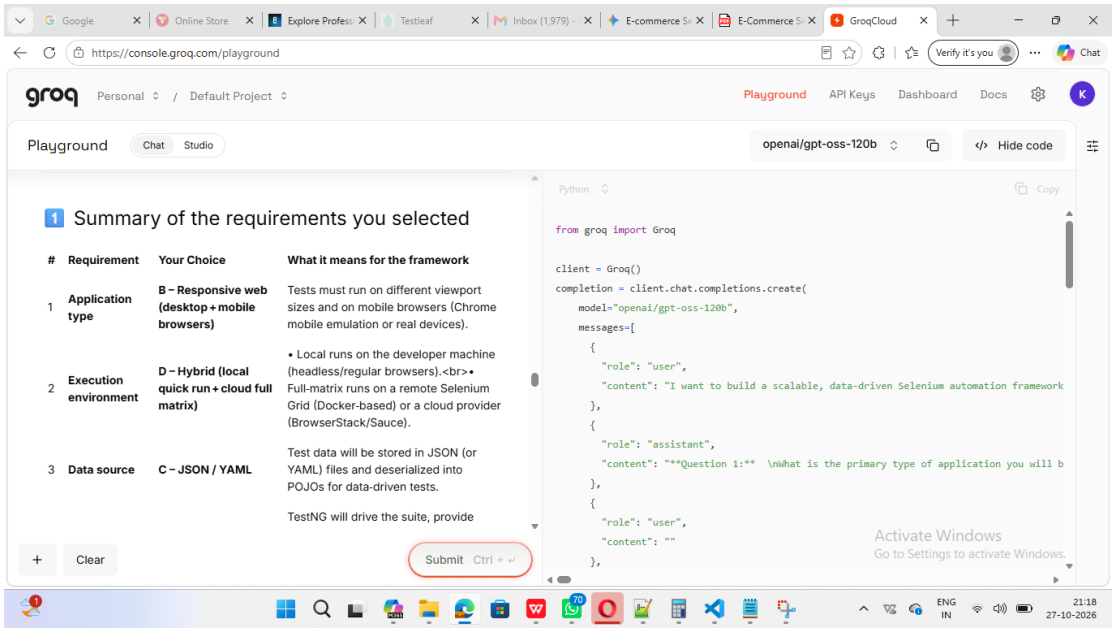
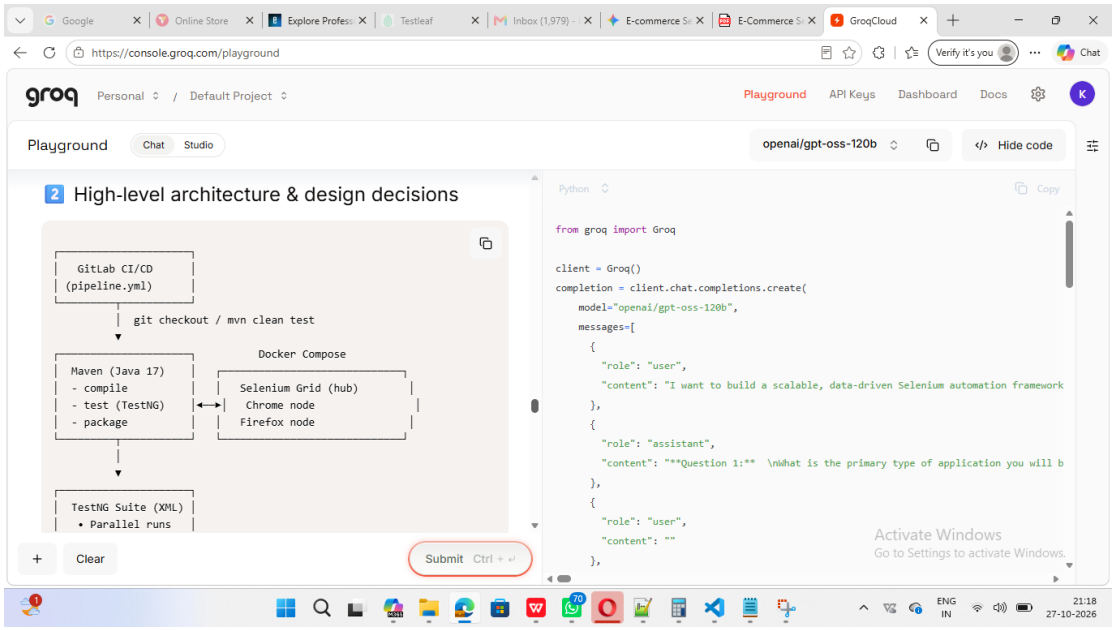
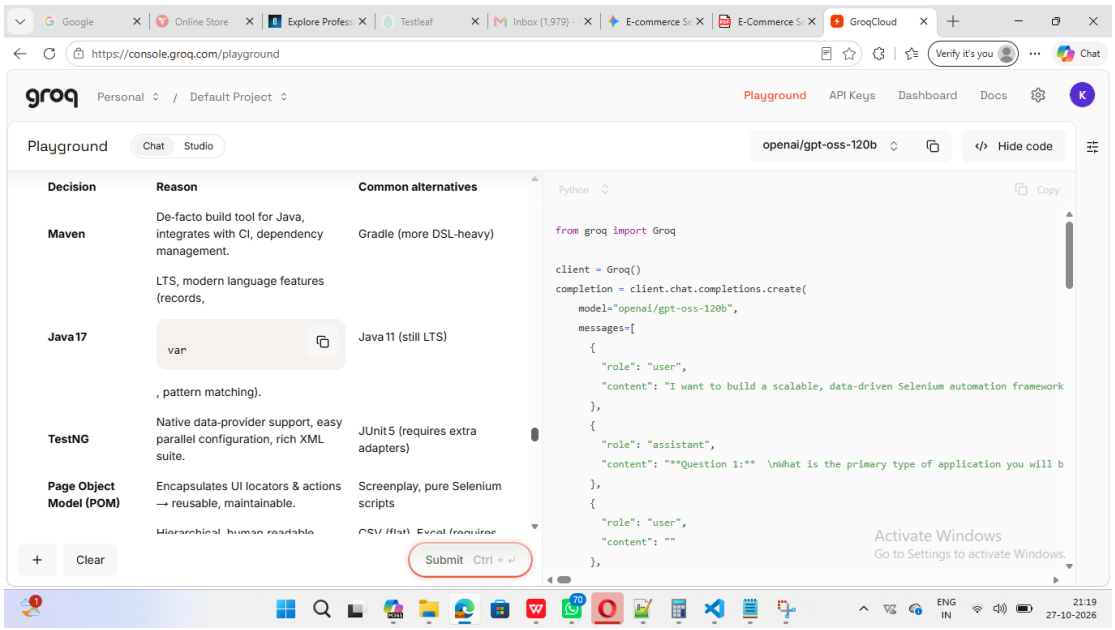

https://console.groq.com/playground
groq
Personal / Default Project
Playground
API Keys
Dashboard
Docs
Playground
Chat
Studio
openai/gpt-oss-120b
Hide code
Provide options.
Question 2:
Which browsers and execution environments do you plan to support?
A. Local desktop browsers only – Tests run on a developer's machine (Chrome, Firefox, Edge).
B. Local + headless mode – Same browsers but also run in headless mode for CI speed-up.
C. Cross-browser cloud services (e.g., Sauce Labs, BrowserStack, AWS Device Farm) – Tests are executed on remote Selenium grids with a wide matrix of browsers/versions.
D. Hybrid approach – Local execution for quick feedback + cloud execution for full-matrix regression runs.
Select the option that matches your intended execution strategy.
USER
Enter user message...
Clear
Submit Ctrl + ↵
Python
Copy
from groq import Groq
client = Groq()
completion = client.chat.completions.create(
model="openai/gpt-oss-120b",
messages=[
{
"role": "user",
"content": "I want to build a scalable, data-driven Selenium automation framework
},
{
"role": "assistant",
"content": "**Question 1:** \nWhat is the primary type of application you will b
},
{
"role": "user",
"content": ""
},
Activate Windows
Go to Settings to activate Windows.
ENG
IN
21:16
27-10-2026

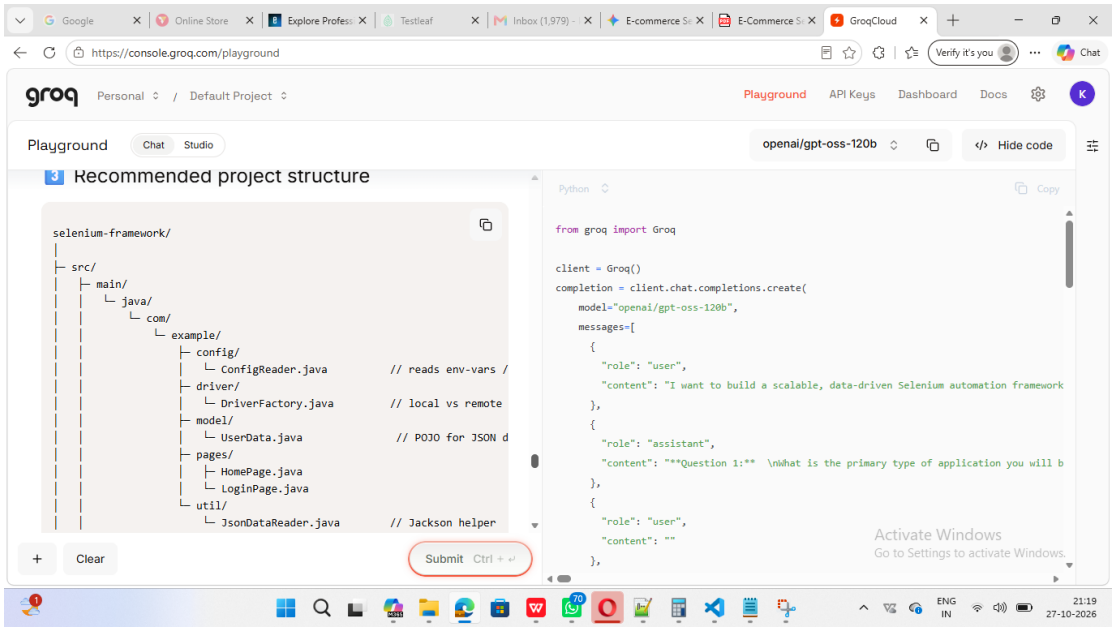

https://console.groq.com/playground
groq
Personal / Default Project
Playground
API Keys
Dashboard
Docs
Playground
Chat
Studio
openai/gpt-oss-120b
Hide code
WebDriverManager, etc.
Proceed with next question.
Question 3:
What will be the primary source of test data for your data-driven scenarios?
A. Excel files (XLS/XLSX) – Familiar to many QA teams; easy to edit manually.
B. CSV files – Simple, lightweight, no external libraries required for reading.
C. JSON or YAML files – Good for hierarchical data and easy to map to POJOs.
D. Database (e.g., MySQL, PostgreSQL) – Real-world data retrieval; useful for large datasets or dynamic data.
Choose the option that best fits your data-driving needs.
USER
C
Latency: 532 ms
Tokens/s: 478
Clear
Submit Ctrl + ↵
Python
Copy
from groq import Groq
client = Groq()
completion = client.chat.completions.create(
model="openai/gpt-oss-120b",
messages=[
{
"role": "user",
"content": "I want to build a scalable, data-driven Selenium automation framework
},
{
"role": "assistant",
"content": "**Question 1:** \nWhat is the primary type of application you will b
},
{
"role": "user",
"content": ""
},
Activate Windows
Go to Settings to activate Windows.
ENG
IN
21:17
27-10-2026

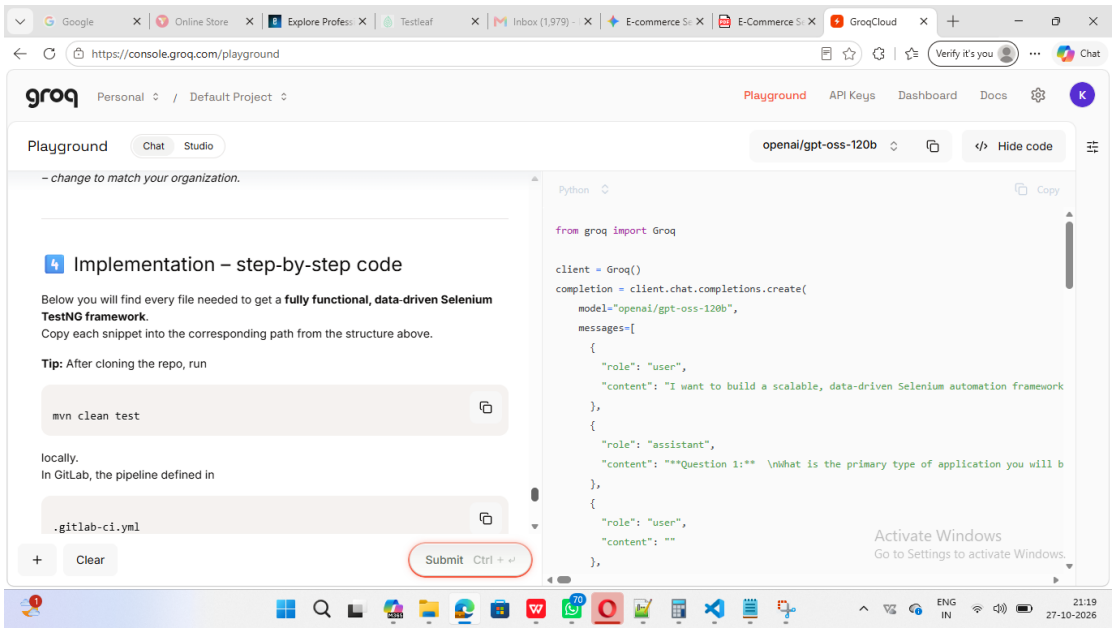

https://console.groq.com/playground
groq
Personal / Default Project
Playground
API Keys
Dashboard
Docs
Playground
Chat
Studio
openai/gpt-oss-120b
Hide code
A. TestNG – Built-in support for data providers, flexible suites, parallel execution, and rich XML configuration.
B. JUnit 5 – Modern JUnit features, but requires additional adapters for TestNG-style data providers.
C. Cucumber (BDD) with TestNG/JUnit – Write tests in Gherkin feature files and map them to step definitions.
D. Hybrid (TestNG for core tests + Cucumber for a subset of BDD scenarios) – Mix both styles in the same project.
*Select the option that aligns with your testing style.*Your turn – please pick the option (A, B, C, or D) that best matches the test execution and reporting framework you'd like to adopt.
USER
A
Latency: 756 ms
Tokens/s: 481
Clear
Submit Ctrl + ↵
Python
Copy
from groq import Groq
client = Groq()
completion = client.chat.completions.create(
model="openai/gpt-oss-120b",
messages=[
{
"role": "user",
"content": "I want to build a scalable, data-driven Selenium automation framework
},
{
"role": "assistant",
"content": "**Question 1:** \nWhat is the primary type of application you will b
},
{
"role": "user",
"content": ""
},
Activate Windows
Go to Settings to activate Windows.
21:17
27-10-2026

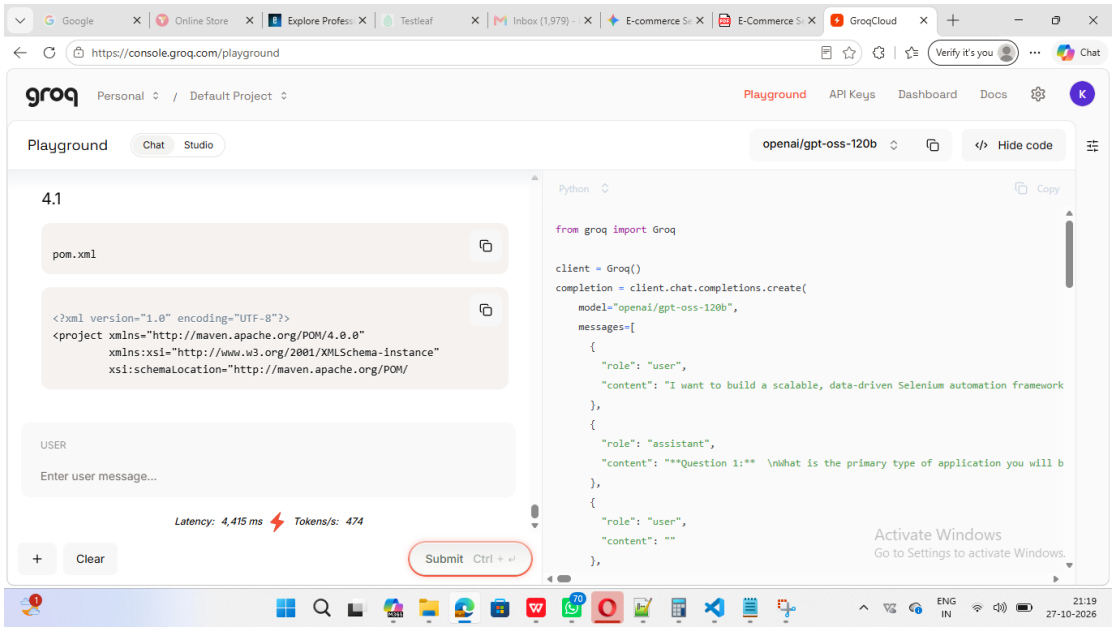

https://console.groq.com/playground
groq
Personal / Default Project
Playground
API Keys
Dashboard
Docs
Playground
Chat
Studio
openai/gpt-oss-120b
Hide code
1 Summary of the requirements you selected
Requirement Your Choice What it means for the framework
1 Application type B – Responsive web (desktop + mobile browsers) Tests must run on different viewport sizes and on mobile browsers (Chrome mobile emulation or real devices).
2 Execution environment D – Hybrid (local quick run + cloud full matrix) • Local runs on the developer machine (headless/regular browsers).
• Full-matrix runs on a remote Selenium Grid (Docker-based) or a cloud provider (BrowserStack/Sauce).
3 Data source C – JSON / YAML Test data will be stored in JSON (or YAML) files and deserialized into POJOs for data-driven tests.
TestNG will drive the suite, provide
Clear
Submit Ctrl + ↵
Python
Copy
from groq import Groq
client = Groq()
completion = client.chat.completions.create(
model="openai/gpt-oss-120b",
messages=[
{
"role": "user",
"content": "I want to build a scalable, data-driven Selenium automation framework
},
{
"role": "assistant",
"content": "**Question 1:** \nWhat is the primary type of application you will b
},
{
"role": "user",
"content": ""
},
Activate Windows
Go to Settings to activate Windows.
21:18
27-10-2026

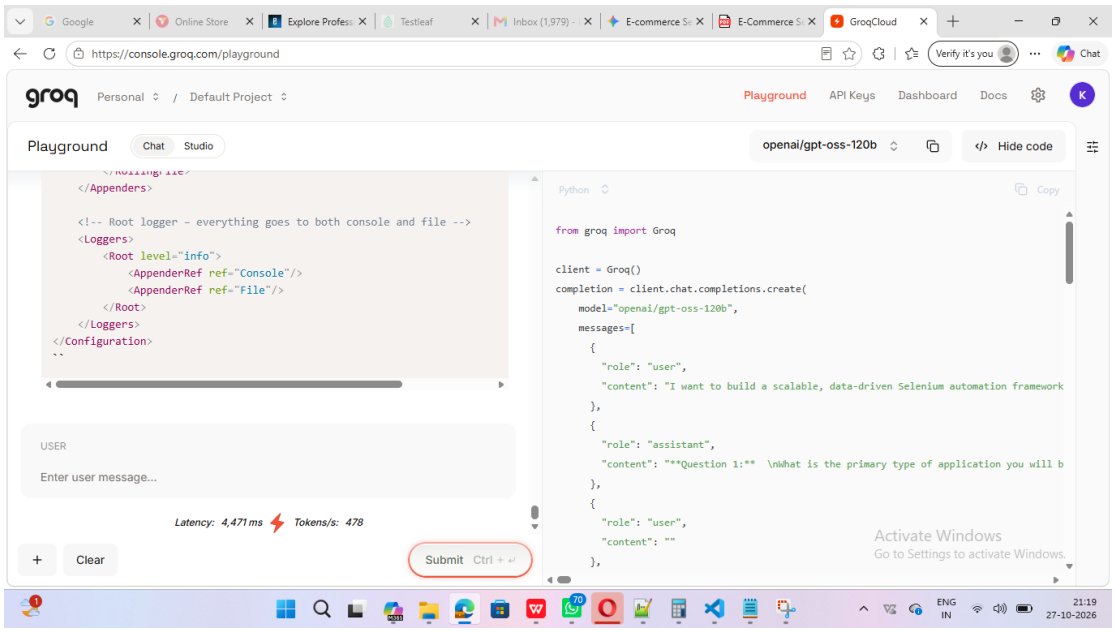

GroqCloud
https://console.groq.com/playground
groq
Personal / Default Project
Playground
API Keys
Dashboard
Docs
Playground
Chat
Studio
openai/gpt-oss-120b
Hide code
2 High-level architecture & design decisions
GitLab CI/CD
(pipeline.yml)
git checkout / mvn clean test
Maven (Java 17)
- compile
- test (TestNG)
- package
Docker Compose
Selenium Grid (hub)
Chrome node
Firefox node
TestNG Suite (XML)
• Parallel runs
Python
Copy
from groq import Groq
client = Groq()
completion = client.chat.completions.create(
model="openai/gpt-oss-120b",
messages=[
{
"role": "user",
"content": "I want to build a scalable, data-driven Selenium automation framework
},
{
"role": "assistant",
"content": "**Question 1:** \nWhat is the primary type of application you will b
},
{
"role": "user",
"content": ""
},
Clear
Submit Ctrl + ↵
Activate Windows
Go to Settings to activate Windows.
ENG
IN
21:18
27-10-2026


GroqCloud
https://console.groq.com/playground
groq
Personal / Default Project
Playground
API Keys
Dashboard
Docs
Playground
Chat
Studio
openai/gpt-oss-120b
Hide code
Decision
Reason
Common alternatives
Maven
De-facto build tool for Java, integrates with CI, dependency management.
Gradle (more DSL-heavy)
Java 17
LTS, modern language features (records,
var
Java 11 (still LTS)
, pattern matching).
TestNG
Native data-provider support, easy parallel configuration, rich XML suite.
JUnit 5 (requires extra adapters)
Page Object Model (POM)
Encapsulates UI locators & actions → reusable, maintainable.
Screenplay, pure Selenium scripts
Python
Copy
from groq import Groq
client = Groq()
completion = client.chat.completions.create(
model="openai/gpt-oss-120b",
messages=[
{
"role": "user",
"content": "I want to build a scalable, data-driven Selenium automation framework
},
{
"role": "assistant",
"content": "**Question 1:** \nWhat is the primary type of application you will b
},
{
"role": "user",
"content": ""
},
Clear
Submit Ctrl + ↵
Activate Windows
Go to Settings to activate Windows.
ENG
IN
21:19
27-10-2026


https://console.groq.com/playground
groq
Personal / Default Project
Playground
API Keys
Dashboard
Docs
Playground
Chat
Studio
openai/gpt-oss-120b
Hide code
Recommended project structure
selenium-framework/
├ src/
│ ├ main/
│ │ └ java/
│ │ └ com/
│ │ └ example/
│ │ ├ config/
│ │ │ └ ConfigReader.java // reads env-vars /
│ │ ├ driver/
│ │ │ └ DriverFactory.java // local vs remote
│ │ ├ model/
│ │ │ └ UserData.java // POJO for JSON d
│ │ ├ pages/
│ │ │ ├ HomePage.java
│ │ │ └ LoginPage.java
│ │ └ util/
│ │ └ JsonDataReader.java // Jackson helper
Python
Copy
from groq import Groq
client = Groq()
completion = client.chat.completions.create(
model="openai/gpt-oss-120b",
messages=[
{
"role": "user",
"content": "I want to build a scalable, data-driven Selenium automation framework
},
{
"role": "assistant",
"content": "**Question 1:** \nWhat is the primary type of application you will b
},
{
"role": "user",
"content": ""
},
Clear
Submit Ctrl + ↵
Activate Windows
Go to Settings to activate Windows.
ENG
IN
21:19
27-10-2026


https://console.groq.com/playground
groq
Personal / Default Project
Playground
API Keys
Dashboard
Docs
Playground
Chat
Studio
openai/gpt-oss-120b
Hide code
– change to match your organization.
Implementation – step-by-step code
Below you will find every file needed to get a fully functional, data-driven Selenium TestNG framework.
Copy each snippet into the corresponding path from the structure above.
Tip: After cloning the repo, run
mvn clean test
locally.
In GitLab, the pipeline defined in
.gitlab-ci.yml
Python
Copy
from groq import Groq
client = Groq()
completion = client.chat.completions.create(
model="openai/gpt-oss-120b",
messages=[
{
"role": "user",
"content": "I want to build a scalable, data-driven Selenium automation framework
},
{
"role": "assistant",
"content": "**Question 1:** \nWhat is the primary type of application you will b
},
{
"role": "user",
"content": ""
},
Clear
Submit Ctrl + ↵
Activate Windows
Go to Settings to activate Windows.
ENG
IN
21:19
27-10-2026


https://console.groq.com/playground
groq
Personal / Default Project
Playground
API Keys
Dashboard
Docs
Playground
Chat
Studio
openai/gpt-oss-120b
Hide code
4.1
pom.xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/
USER
Enter user message...
Latency: 4,415 ms
Tokens/s: 474
Clear
Submit Ctrl + ↵
Python
Copy
from groq import Groq
client = Groq()
completion = client.chat.completions.create(
model="openai/gpt-oss-120b",
messages=[
{
"role": "user",
"content": "I want to build a scalable, data-driven Selenium automation framework
},
{
"role": "assistant",
"content": "**Question 1:** \nWhat is the primary type of application you will b
},
{
"role": "user",
"content": ""
},
Activate Windows
Go to Settings to activate Windows.


https://console.groq.com/playground
groq
Personal / Default Project
Playground
API Keys
Dashboard
Docs
Playground
Chat
Studio
openai/gpt-oss-120b
Hide code
</Appenders>
<!-- Root logger - everything goes to both console and file -->
<Loggers>
<Root level="info">
<AppenderRef ref="Console"/>
<AppenderRef ref="File"/>
</Root>
</Loggers>
</Configuration>
USER
Enter user message...
Latency: 4,471 ms
Tokens/s: 478
Clear
Submit Ctrl + ↵
Python
Copy
from groq import Groq
client = Groq()
completion = client.chat.completions.create(
model="openai/gpt-oss-120b",
messages=[
{
"role": "user",
"content": "I want to build a scalable, data-driven Selenium automation framework
},
{
"role": "assistant",
"content": "**Question 1:** \nWhat is the primary type of application you will b
},
{
"role": "user",
"content": ""
},
Activate Windows
Go to Settings to activate Windows.